EduGenie

AI-Powered Educational Assistant
Full Project Presentation • 30 Pages

Role	Name
Team Leader	Vishnu P
Team Member	Abinesh B
Team Member	Vignesh V
Team Member	Giri S

Project Overview

- EduGenie is a Generative AI based educational assistant for students.
- It provides quick, student-friendly answers through a simple web interface.
- The application connects a frontend, Flask backend and Gemini AI service.
- Designed as a practical full-stack academic project.

Problem Statement

- Students often need quick explanations while studying.
- Searching across many resources can take time.
- Answers may be difficult to understand or too detailed.
- A simple AI assistant can provide focused learning support.

Project Objectives

- Provide quick answers to educational questions.
- Make AI-based learning assistance simple and accessible.
- Support explanations, summaries, quizzes and recommendations.
- Demonstrate integration of web development with Generative AI.
- Create a project that can be extended with future learning features.

Scope of the Project

- Question answering for academic topics.
- Student-friendly explanations of concepts.
- Paragraph summarization.
- Quiz generation for self-assessment.
- Learning recommendations based on a topic.

Technologies Used

- Frontend: HTML, CSS and JavaScript
- Backend: Python with Flask
- Generative AI: Google Gemini API
- Configuration: environment variables / .env
- Development: Visual Studio Code
- Testing: Pytest

System Architecture



1. User enters a question → 2. Frontend sends request → 3. Flask calls Gemini → 4. Answer returns to the browser

Application Workflow

- Open the EduGenie web application.
- Enter an educational question in the input field.
- Click Get Answer.
- JavaScript sends the request to the Flask backend.
- Flask forwards the prompt to Gemini AI.
- The generated response is displayed in the interface.

Actual Application – Main Screen

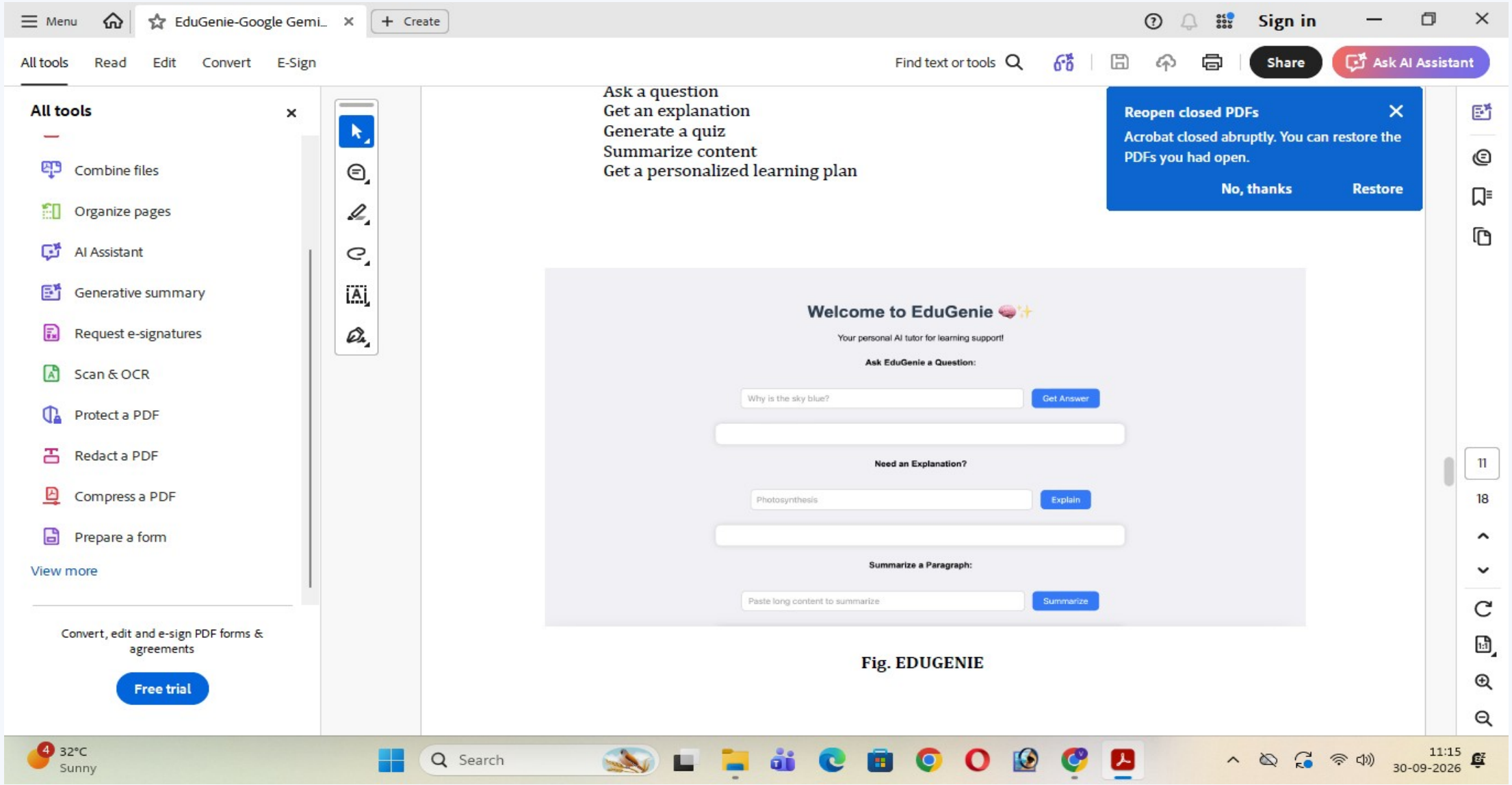


Fig. EDUGENIE

Frontend: HTML Structure


```
<!DOCTYPE html>
<html>
<head>
  <title>EduGenie</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
  <div class="container">
    <h1>EduGenie</h1>
    <input id="question" placeholder="Ask your question">
    <button onclick="askQuestion()">Get Answer</button>
    <div id="answer"></div>
  </div>
  <script src="/static/script.js"></script>
</body>
</html>
```

Frontend: JavaScript Request


```
async function askQuestion() {
  const question = document.getElementById("question").value;

  const response = await fetch("/ask", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({question: question})
  });

  const data = await response.json();
  document.getElementById("answer").innerText = data.answer;
}
```

Frontend: CSS Styling


```
.container {
  max-width: 700px;
  margin: 60px auto;
  padding: 30px;
  text-align: center;
}

input {
  width: 80%;
  padding: 12px;
}

button {
  padding: 12px 20px;
  cursor: pointer;
}

#answer {
  margin-top: 25px;
  text-align: left;
}
```

Backend: Flask Application


```
from flask import Flask, request, jsonify, render_template
import os

app = Flask(__name__)

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/ask", methods=["POST"])
def ask():
    data = request.get_json()
    question = data.get("question", "")
    answer = get_gemini_answer(question)
    return jsonify({"answer": answer})

if __name__ == "__main__":
    app.run(debug=True)
```

Backend: Gemini API Function


```
import google.generativeai as genai

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
model = genai.GenerativeModel("gemini-1.5-flash")

def get_gemini_answer(question):
    prompt = f"""Answer this for a student
    in a clear and simple way:
    {question}"""
    response = model.generate_content(prompt)
    return response.text
```

Configuration: Environment Variable


```
# .env
GEMINI_API_KEY=your_api_key_here

# Python
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv("GEMINI_API_KEY")

# Never hard-code or publish your API key.
```

Project Folder Structure

- edugenie/
- app.py — Flask backend and API routes
- templates/index.html — web interface
- static/style.css — page styling
- static/script.js — frontend request handling
- .env — private API configuration
- tests/ — automated test cases

How the Backend Processes a Question



Main Features

- Ask a Question — generates an AI answer.
- Get an Explanation — explains concepts in simple language.
- Summarize — converts longer text into concise points.
- Generate a Quiz — supports practice and self-assessment.
- Learning Recommendations — suggests useful next steps.

Actual Application – Additional Features

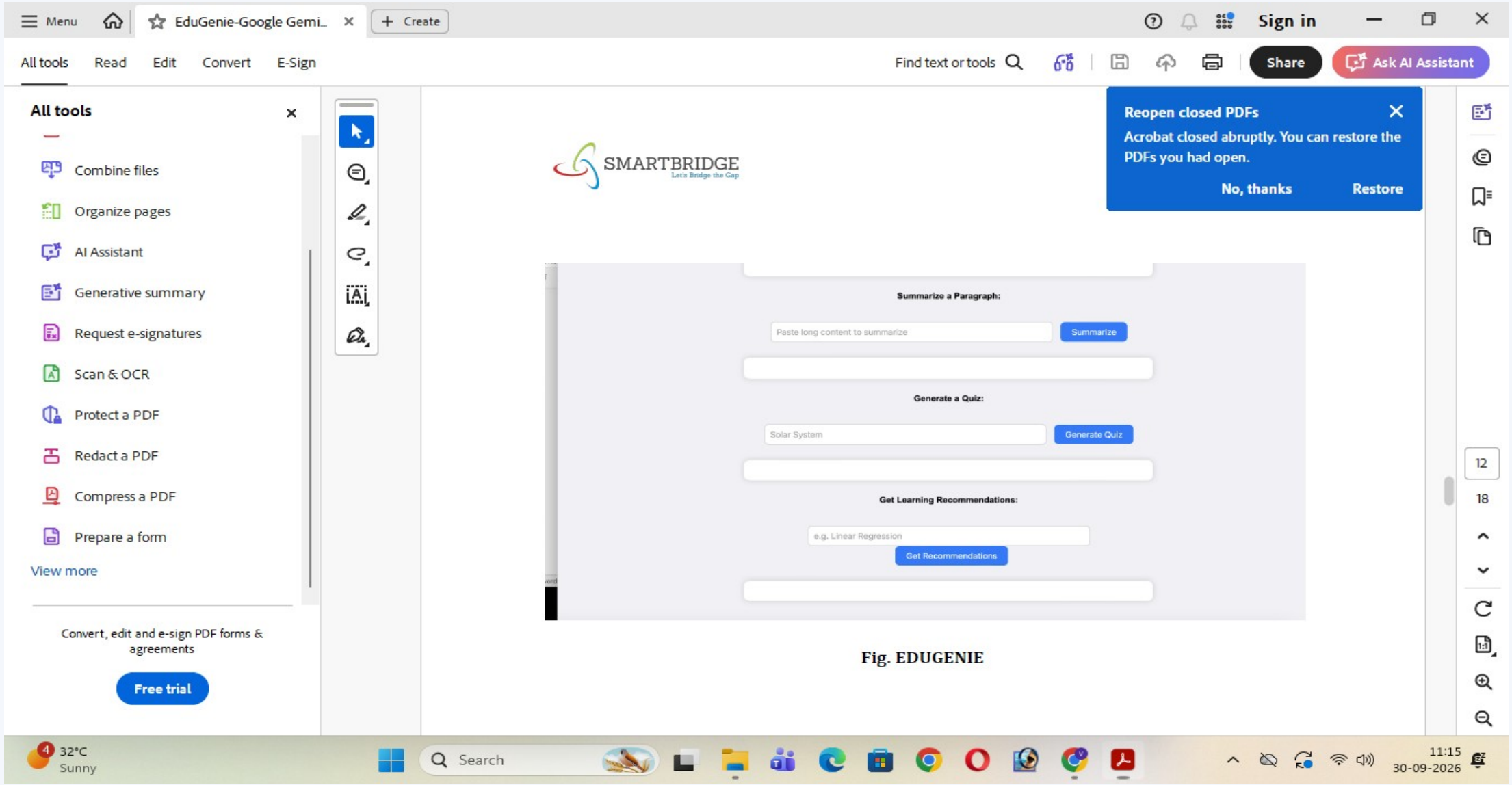
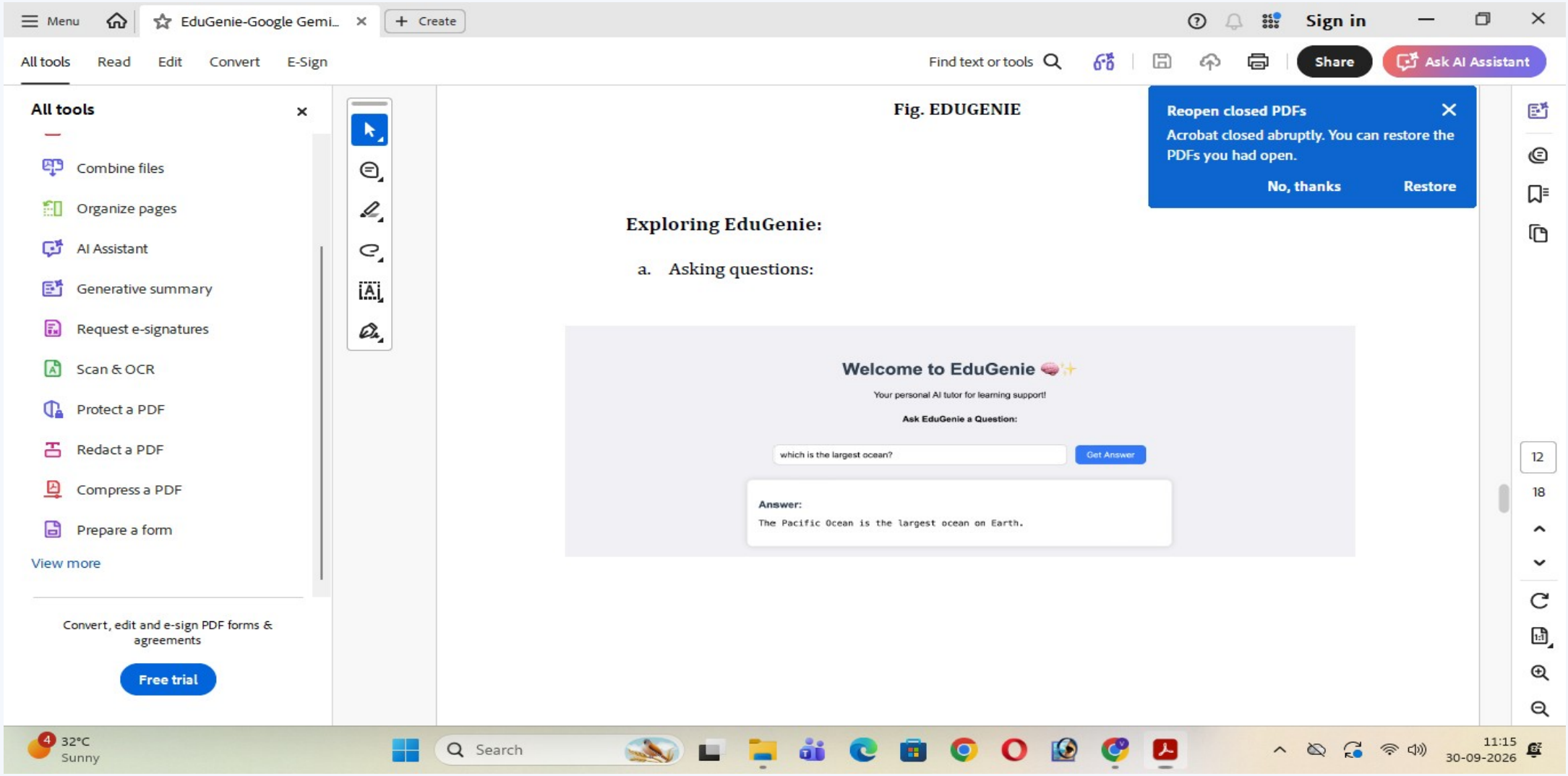


Fig. EDUGENIE

Sample Input and Output



Input / Output Explanation

- INPUT: “Which is the largest ocean?”
- REQUEST: The question is sent to the /ask endpoint.
- PROCESSING: Flask sends the prompt to Gemini AI.
- OUTPUT: “The Pacific Ocean is the largest ocean on Earth.”
- The response is returned as JSON and displayed in the browser.

Prompt Design


```
prompt = f"""
You are EduGenie, a helpful educational assistant.
Answer the student's question clearly.
Use simple language and short explanations.
If useful, include examples or steps.

Student question:
{question}
"""
```

Error Handling


```
@app.route("/ask", methods=["POST"])
def ask():
    try:
        data = request.get_json()
        question = data.get("question", "").strip()

        if not question:
            return jsonify({"error": "Enter a question"}), 400

        answer = get_gemini_answer(question)
        return jsonify({"answer": answer})

    except Exception as e:
        return jsonify({"error": "Unable to process request"}), 500
```

Testing Strategy

- Open the application in a browser.
- Test valid educational questions.
- Test empty input and invalid requests.
- Verify that answers appear below the input.
- Test explanation, summary and quiz features.
- Run automated tests using Pytest where applicable.

Sample Test Code


```
def test_home_page(client):
    response = client.get("/")
    assert response.status_code == 200

def test_empty_question(client):
    response = client.post(
        "/ask",
        json={"question": ""}
    )
    assert response.status_code == 400
```

Advantages

- Fast access to learning support.
- Simple and user-friendly interface.
- Generative AI can handle many topics.
- Modular architecture makes the project extendable.
- Useful demonstration of full-stack development and AI integration.

Limitations

- AI responses may occasionally contain inaccurate information.
- Internet access is required for the cloud AI service.
- API usage may be subject to provider limits or cost.
- Student data and prompts should be handled carefully.
- The current version can be improved with stronger validation and personalization.

Future Enhancements

- Student login and personalized profiles.
- Chat history and saved answers.
- Subject-wise learning modes.
- Multilingual support.
- Automatic notes and PDF generation.
- Progress tracking and learning analytics.
- Teacher dashboard and quiz result tracking.

Project Result & Conclusion

- EduGenie successfully demonstrates an AI-powered educational web application.
- The frontend provides a simple interaction for students.
- The Flask backend connects the application to Gemini AI.
- The project supports question answering and can be extended with more learning tools.
- It is a practical example of combining Python, web technologies and Generative AI.

Thank You

EduGenie • AI-Powered Educational Assistant
Questions & Discussion